

HATS: Hardness-Aware Trajectory Synthesis for GUI Agents

Rui Shao^{1,3,†}, Ruize Gao^{2,†}, Bin Xie¹, Yixing Li¹,
Kaiwen Zhou⁴, Shuai Wang⁴, Weili Guan^{1,3}, Gongwei Chen^{1,*}

¹Harbin Institute of Technology, Shenzhen ²National University of Singapore, CNRS@CREATE

³Shenzhen Loop Area Institute ⁴Huawei Noah's Ark Lab

shaorui@hit.edu.cn ruizegao@u.nus.edu chengongwei@hit.edu.cn

<https://github.com/JiuTian-VL/HATS>

Abstract

Graphical user interface (GUI) agents powered by large vision-language models (VLMs) have shown remarkable potential in automating digital tasks, highlighting the need for high-quality trajectory data to support effective agent training. Yet existing trajectory synthesis pipelines often yield agents that fail to generalize beyond simple interactions. We identify this limitation as stemming from the neglect of semantic-ambiguous actions—interactions whose meanings are context-dependent, sequentially dependent, or visually ambiguous. Such actions are crucial for real-world robustness but are under-represented and poorly processed in current datasets, leading to semantic misalignment between task instructions and execution. To address these issues, we propose **HATS**, a **Hardness-Aware Trajectory Synthesis** framework designed to mitigate the impact of semantic ambiguity. We define hardness as the degree of semantic ambiguity associated with an action and develop two complementary modules: (1) a hardness-driven exploration that guides data collection toward ambiguous yet informative interactions, and (2) an alignment-guided refinement that iteratively validates and repairs instruction-execution alignment. The two modules operate in a closed-loop manner—exploration supplies refinement with challenging trajectories, while refinement feedback updates the hardness signal to guide future exploration. Extensive experiments show that agents trained with **HATS** consistently outperform state-of-the-art baselines across benchmark GUI environments.

1. Introduction

Graphical User Interface (GUI) agents powered by large **Vision-Language Models (VLMs)** have emerged as a promising direction for automating complex digital work-

[†]Equal contribution ^{*}Corresponding author

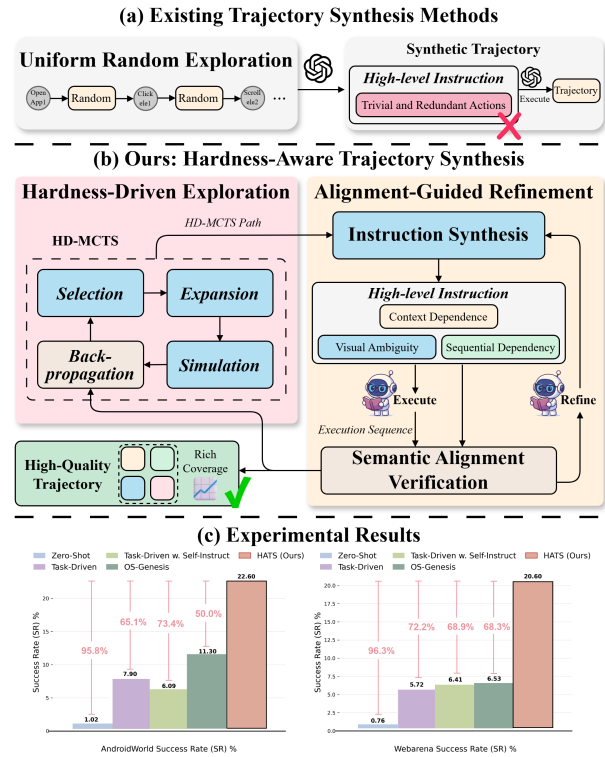


Figure 1. Overview of trajectory synthesis paradigms. Compared with (a) existing methods, (b) **HATS** integrates hardness-driven exploration and alignment-guided refinement in a closed loop, producing high-quality trajectories with rich semantic coverage and strong instruction-execution alignment. (c) Experiments show **HATS** outperforms OS-GENESIS by **100%↑** on ANDROIDWORLD (22.60 vs. 11.30) and **215%↑** on WEBARENA (20.60 vs. 6.53).

flows, from booking travel in mobile apps to configuring cloud services in browsers. A capable agent must perceive the screen, interpret a natural-language goal, decompose it into a plan, and execute actions such as TAP, SCROLL, or TYPE. Training such autonomy requires instruction-aligned

trajectories coupling task goals, step-level descriptions, and exact action–state pairs [77]. However, collecting such trajectories manually is prohibitively expensive, motivating recent efforts toward automated trajectory synthesis.

Automating Trajectory Synthesis. To reduce human annotation, recent work automates trajectory generation. Early **task-driven** approaches begin with written goals and record agents or humans executing them [25, 27, 37], but still depend on costly goal specification and yield inconsistent traces [39, 44]. OS-GENESIS [59] instead introduces **reverse task synthesis**, where the system explores applications via random UI actions and then synthesizes task instructions to describe observed outcomes, improving diversity by removing manual goal authoring.

The Semantic-Ambiguity Gap. Despite these advantages, reverse task synthesis [59] still faces critical limitations. Exploration based on uniform random walks or shallow breadth-first search causes over 70% of collected traces to collapse into trivial **semantic-intuitive actions** (e.g., “open menu”, “tap back”), which quickly saturate learning yet dominate storage and training budgets. Conversely, the resulting corpora remain deficient in **semantic-ambiguous actions**—UI interactions whose functional meaning depends on subtle visual or contextual cues and are easily confused or hidden.

As illustrated in Fig. 2, these actions include (a) identical “plus” icons that trigger different functions depending on context, (b) operations that only succeed after prerequisite steps, and (c) visually similar elements leading to distinct outcomes. Such rare yet crucial actions represent the key bottlenecks where agents fail to generalize: without adequate exposure, they plateau at superficial behaviors and cannot handle unseen workflows. Moreover, when these actions do appear, their ambiguity often leads to **instruction–execution misalignment**: synthesized instructions may omit essential context or replay inconsistently, introducing noisy supervision that further degrades data quality. This issue is exacerbated by existing one-shot instruction generation methods [59], which lack semantic validation and thus produce corpora that are both redundant and unreliable. The **challenge** is thus twofold: (i) how to focus exploration on high-value, semantically ambiguous regions of the state space rather than collecting redundant, intuitive actions; and (ii) how to ensure alignment between synthesized instructions and actual executions to prevent supervision noise. Existing one-shot instruction generation lacks semantic validation, making the resulting corpora both redundant and unreliable.

Hardness-Aware Trajectory Synthesis (HATS). To address these challenges, we propose **HATS**, a closed-loop trajectory synthesis framework explicitly targeting semantic-ambiguous actions through two cooperative modules (Fig. 1). The **Hardness-Driven Exploration** module (§3.1) steers search toward semantically complex, under-represented

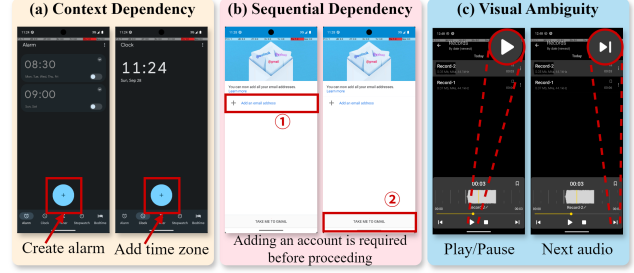


Figure 2. Illustrative cases of semantic-ambiguous actions. Such actions constitute critical bottlenecks for robust agent generalization but are rarely captured by existing synthesis pipelines.

states, while the **Alignment-Guided Refinement** module (§3.2) performs multi-round replay and repair to ensure semantic consistency before data admission. Both modules operate under a unified **Hardness-Driven Monte Carlo Tree Search (HD-MCTS)** procedure, which orchestrates exploration, refinement, and feedback propagation in a single control loop. Exploration supplies challenging trajectories for validation, and refinement converts misalignment feedback into hardness signals that guide subsequent exploration. This closed-loop design systematically enhances both the diversity and fidelity of synthesized data, yielding more reliable and generalizable GUI agents.

Contributions. We summarize our contributions as follows:

- **I.** We introduce **HATS**, a unified closed-loop trajectory synthesis framework that enhances the representation and handling of semantic-ambiguous actions.
- **II.** We design a hardness-driven exploration module that prioritizes rare, high-value actions while reducing redundancy from intuitive ones.
- **III.** We propose an alignment-guided refinement module that replaces one-shot synthesis with multi-round validation, ensuring instruction–execution alignment.

These contributions collectively establish a scalable, semantically grounded pipeline that enhances diversity and fidelity in trajectory synthesis, advancing GUI agent training.

2. Related Works

GUI Agents. Recent advances in large language and vision–language models (LLMs/VLMs) have accelerated progress in agents that perceive, reason, and act within graphical user interfaces [11, 12, 65]. Earlier systems relied on prompt programming or external tool execution [58, 64], while more recent work fine-tunes multimodal backbones to better recognize UI states [5, 66] and execute atomic actions across web, desktop, and mobile environments [23, 27, 60]. Although these agents demonstrate strong generality, their performance remains heavily constrained by the quality and diversity of available trajectory data, especially when handling semantically complex interactions.

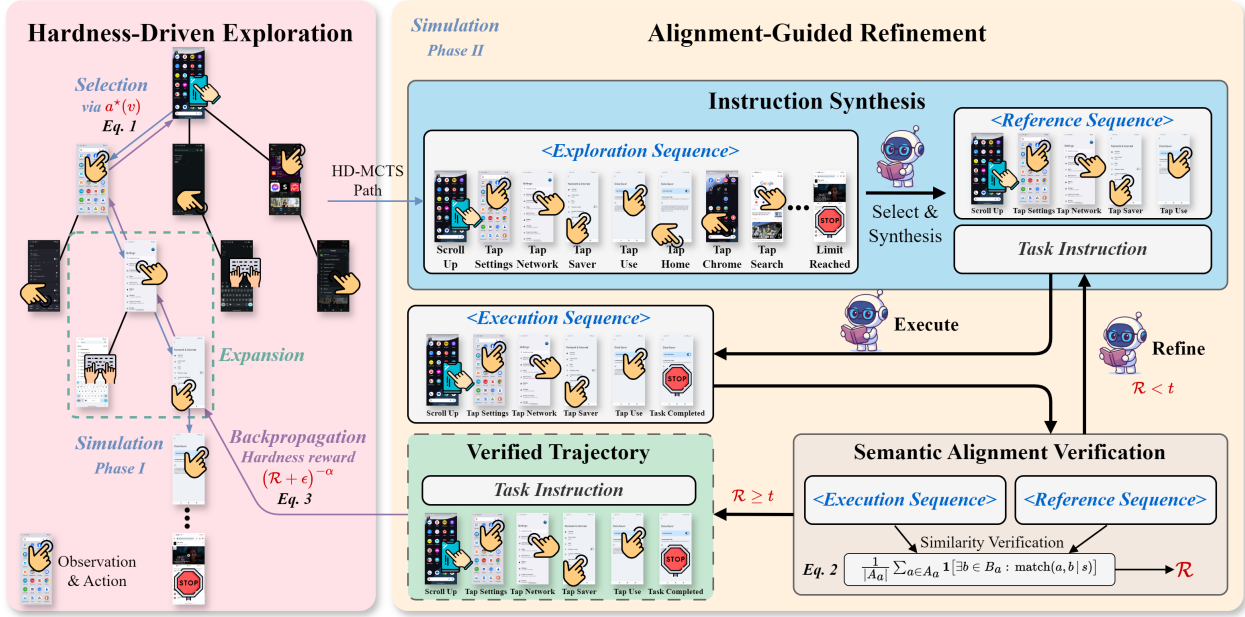


Figure 3. **Architecture of the HATS framework.** The framework integrates a **Hardness-Driven Exploration** module (§3.1) and an **Alignment-Guided Refinement** module (§3.2) within a unified **HD-MCTS** loop. Exploration corresponds to the **Selection**, **Expansion**, and **Simulation Phase I**, while refinement handles **Simulation Phase II** and **Backpropagation**. Misalignment detected during refinement is converted into a hardness reward that guides subsequent exploration, forming a closed loop for progressively improving both diversity and semantic fidelity of synthesized trajectories.

GUI Trajectory Synthesis. A major challenge in data construction is obtaining large-scale, instruction-aligned trajectories that reflect realistic interface behaviors [8, 48, 54, 78]. Existing approaches follow two paradigms. **TASK-DRIVEN** demonstrations [27, 37] require manually authored goals and human/model executions, but are expensive and often yield inconsistent traces [39, 44]. To improve scalability, **OS-GENESIS** [59] adopts an **exploration-driven** strategy, exploring UIs first and then synthesizing task instructions from observed outcomes. However, naive exploration oversamples repetitive **semantic-intuitive actions** while underrepresenting **semantic-ambiguous actions** whose functional meaning depends on contextual, sequential, or visual cues. Moreover, one-shot reverse synthesis frequently produces underspecified instructions that fail to replay, leading to **instruction-execution misalignment**. These limitations motivate our hardness-aware, closed-loop framework, which targets ambiguous interactions during exploration and verifies synthesized instructions through iterative alignment. Other contemporaneous efforts explore stochastic or intent-aware multi-app exploration [33], which is largely orthogonal to our focus on addressing semantic ambiguity through hardness-aware sampling and alignment refinement.

3. Hardness-Aware Trajectory Synthesis

Existing automated pipelines struggle in two key ways: (i) they rarely capture semantic-ambiguous actions, i.e., inter-

actions whose semantics depend on contextual, sequential, or visual cues; and (ii) even when such actions are captured, one-shot instruction generation often produces noisy or misaligned descriptions that degrade supervision quality. Our framework **HATS** addresses both issues by tightly coupling targeted exploration with alignment-aware data verification.

Method Overview. As illustrated in Fig. 3, **HATS** consists of two interacting modules: a **Hardness-Driven Exploration** module that actively seeks semantically ambiguous, high-value actions (§3.1), and an **Alignment-Guided Refinement** module that validates and repairs the resulting instruction–trajectory pairs before admitting them into the training corpus (§3.2). These two modules are fused through a customized **Hardness-Driven Monte Carlo Tree Search** (HD-MCTS) procedure that forms the unified control loop of our framework. Instead of treating exploration and supervision cleanup as two separate offline stages, **HATS** unifies them into a single iterative search process. Each HD-MCTS iteration (Appendix Alg. 1) performs four coordinated stages:

- **Selection:** the agent traverses an action tree to balance exploitation and exploration using a UCB policy;
- **Expansion:** the tree grows by executing previously unvisited actions and adding corresponding nodes;
- **Simulation:** the agent rolls out a bounded-depth trajectory, reverse-synthesizes a natural-language instruction, replays and **iteratively refines** that instruction until it

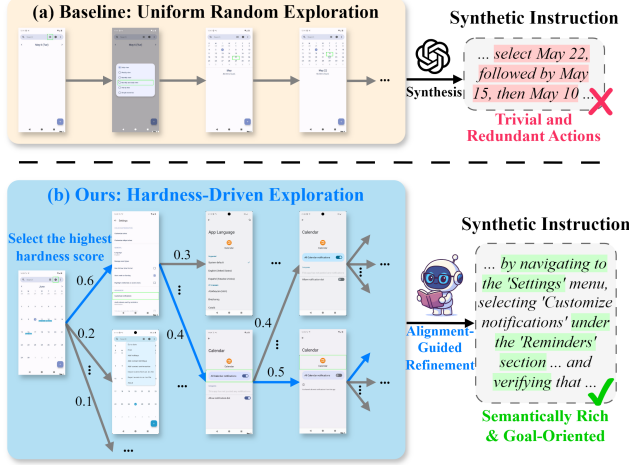


Figure 4. Comparison of exploration strategies. **Uniform Random Exploration** [59] often yields trivial and redundant actions, whereas our **Hardness-Driven Exploration** replaces random walking with a hardness-driven exploration policy that selectively targets under-represented yet semantically challenging actions.

becomes executable and semantically aligned;

- **Backpropagation:** an **alignment-derived hardness reward** is propagated up the tree to bias future exploration toward semantically challenging regions.

Notations. Let $\mathcal{E}$ denote a GUI environment. At time t , the agent observes a state s_t (screenshot + optional UI tree), executes an action $a_t \in \mathcal{A}$, and transitions to $s_{t+1} = \mathcal{E}(s_t, a_t)$. The atomic action set is $\mathcal{A} = \{\text{TAP, TYPE, SCROLL, BACK, LONG-PRESS, SWIPE}\}$. A trajectory $\tau = (s_0, a_0, s_1, \dots, s_T)$ records a sequence of interactions. We represent $\mathcal{E}$ as an **action tree**, where each node v corresponds to a state $s(v)$ and each edge (v, a) maintains a value estimate $Q(v, a)$ and visit count $N(v, a)$, with $N(v) = \sum_a N(v, a)$. $\mathcal{P}$ denotes the current path (a sequence of (a, s) pairs) with rollout depth limit $T_{\max}$.

3.1. Hardness-Driven Exploration Module

We follow the **interaction-driven discovery paradigm** introduced by OS-GENESIS [59], where the agent explores by performing UI actions rather than following pre-defined goals. As contrasted in Fig. 4, however, OS-GENESIS performs **reverse task synthesis** from randomly sampled traces, often yielding trivial and redundant actions. In contrast, our approach replaces random walking with a **hardness-driven exploration policy** that selectively targets under-represented yet semantically challenging actions. This module corresponds to the **Selection** and **Expansion** phases, as well as **Simulation Phase I** in HD-MCTS.

I. Selection on the Action Tree. Starting from the root node v_0 with the initial GUI state s_0 , HD-MCTS traverses the action tree by recursively selecting outgoing edges using an

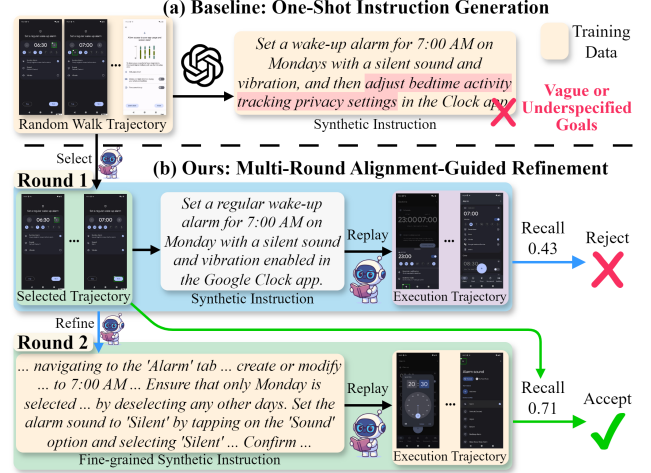


Figure 5. Comparison of instruction synthesis methods. **One-Shot Instruction Generation** [59] directly maps raw traces to text, often yielding vague or underspecified goals and inconsistent executions. In contrast, our **Multi-Round Alignment-Guided Refinement** iteratively replays and verifies task instructions to produce semantically faithful and executable Verified Trajectories.

Upper Confidence Bound (UCB) rule:

$$a^*(v) = \arg \max_{a \in \mathcal{A}(v)} Q(v, a) + C \sqrt{\frac{\ln(N(v)+1)}{N(v, a)+1}}, \quad (1)$$

where C controls exploration pressure. Unlike classical MCTS, $Q(v, a)$ is updated via the **hardness reward** (Eq. 3), which we formally define later in the closed-loop integration stage. This reward serves as an innovative bridge between exploration and semantic feedback—transforming alignment hardness into a guiding signal that steers future search toward more informative, ambiguous behaviors.

II. Expansion. If the current node v is not fully expanded, we select an unvisited action $a_e \in \mathcal{A}(v)$, apply it to obtain $s_e = \mathcal{E}(s(v), a_e)$, create a child node u with state $s(u) = s_e$, and append (a_e, s_e) to $\mathcal{P}$. This grows the search frontier along under-explored branches and increases coverage of semantically diverse UI states.

III. Simulation Phase I: Rollout. After selection and expansion, we perform a bounded-depth **rollout** from the current node by executing valid UI actions until $\mathcal{P}$ reaches depth $T_{\max}$:

$$\mathcal{P} \leftarrow \text{ROLLOUT}(\mathcal{E}, v, s, \mathcal{P}, T_{\max}).$$

This process produces a concrete multi-step interaction trace, a candidate trajectory, which is then passed to the **alignment-guided refinement module** for subsequent processing.

3.2. Alignment-Guided Refinement Module

This module corresponds to **Simulation Phase II** in HD-MCTS (see Fig. 3). It receives noisy, partially meaningful

Table 1. **AndroidWorld** evaluation results across task categories. All values are success rates (%). DLS = Daily Life & Services; S&C = Social & Communication; S&U = System & Utility; P&W = Productivity & Work. The final column shows overall success rate.

Base Model	Strategies	Task Categories (%)				Overall SR (%)
		DLS	S&C	S&U	P&W	
GPT-4o [20]	Zero-Shot [63]	42.86 N.A.	66.67 N.A.	77.27 N.A.	29.09 N.A.	45.22 N.A.
InternVL2-4B [4]	Zero-Shot [63]	0.00 -100.0%	0.00 -100.0%	2.38 -94.2%	0.00 -100.0%	1.02 -95.8%
	Task-Driven [25]	3.49 -79.8%	0.00 -100.0%	19.70 -51.8%	3.70 -57.2%	7.90 -65.1%
	Task-Driven w. Self-Instruct [62]	2.30 -86.7%	3.70 -90.4%	21.21 -48.2%	0.62 -92.8%	6.09 -73.4%
	OS-Genesis [59]	7.14 -58.6%	11.11 -71.2%	36.36 -11.1%	3.64 -57.9%	11.30 -50.0%
HATS		17.24 0.0%	38.46 0.0%	40.91 0.0%	8.64 0.0%	22.60 0.0%
InternVL2-8B [4]	Zero-Shot [63]	0.00 -100.0%	0.00 -100.0%	3.13 -94.1%	0.00 -100.0%	1.04 -95.8%
	Task-Driven [25]	1.15 -92.9%	0.00 -100.0%	12.12 -77.2%	0.62 -95.4%	3.48 -85.7%
	Task-Driven w. Self-Instruct [62]	5.95 -63.1%	0.00 -100.0%	13.64 -74.3%	3.09 -77.2%	6.25 -72.4%
	OS-Genesis [59]	13.79 -14.3%	0.00 -100.0%	47.62 -10.2%	4.08 -70.0%	14.81 -39.4%
HATS		16.09 0.0%	25.93 0.0%	53.03 0.0%	13.58 0.0%	24.35 0.0%
Qwen2-VL-7B [61]	Zero-Shot [63]	0.00 -100.0%	0.00 -100.0%	7.58 -85.3%	0.00 -100.0%	1.75 -92.6%
	Task-Driven [25]	8.33 -54.7%	3.70 -88.0%	30.30 -41.2%	2.47 -75.1%	10.43 -57.2%
	Task-Driven w. Self-Instruct [62]	6.90 -62.5%	7.41 -75.9%	25.76 -50.0%	0.62 -93.7%	6.09 -74.9%
	OS-Genesis [59]	17.86 -3.0%	0.00 -100.0%	31.82 -38.2%	1.85 -81.3%	11.50 -52.7%
HATS		18.39 0.0%	30.77 0.0%	51.52 0.0%	9.88 0.0%	24.35 0.0%

Exploration Sequences from the previous stage and transforms them into executable, instruction-aligned **Verified Trajectory**. Unlike the **One-Shot Instruction Generation** paradigm in OS-GENESIS [59], which directly maps random traces to text, our method employs a **Multi-Round Alignment-Guided Refinement** process that iteratively replays, verifies, and corrects synthesized instructions until semantic and functional consistency are both achieved (Fig. 5).

Prompt-Based Implementation. All stages of refinement that sub-trajectory selection, instruction synthesis, execution replay, similarity verification, and contextual correction are implemented via prompt-driven VLM modules¹.

I. Sub-Trajectory Selection. From the rolled-out **Exploration Sequence** $\mathcal{P}$, we first extract a semantically coherent segment:

$$A \leftarrow \text{SELECTSUBSEQUENCE}(\mathcal{P})^1,$$

representing one meaningful user intent, such as “add a new clock”. This selected segment is referred to as the **Reference Sequence**, serves as the behavioral ground truth for synthesis and verification.

II. Instruction Synthesis. We then generate a natural-language description for the Reference Sequence via

$$I \leftarrow \text{SYNTHESIZEINSTRUCTION}(A)^1,$$

yielding an initial hypothesized instruction. Unlike OS-GENESIS, which treats this output as final, we regard I as provisional and subject to replay-driven verification.

III. Replay and Execution. We execute the synthesized instruction I from the start state of A to obtain an **Execution Sequence**:

$$B \leftarrow \text{EXECUTEINSTRUCTION}(I)^1.$$

This sequence represents how the environment actually re-

sponds to the instruction. Discrepancies between A (expected behavior) and B (observed outcome) indicate semantic gaps in I , e.g., missing contextual cues or ambiguity.

IV. Similarity Verification. To quantitatively measure alignment between the **Reference** and **Execution Sequences**, we perform the similarity verification using the **Action-Level Reconstruction Recall** metric:

$$R(A, B) = \frac{1}{|A_a|} \sum_{a \in A_a} 1[\exists b \in B_a : \text{match}(a, b \mid s)]. \quad (2)$$

Here, $\text{match}(a, b \mid s)^1$ determines whether executing b in state s is semantically equivalent to action a in its original context. A lower $R(A, B)$ implies that the instruction I lacks critical contextual cues, for instance, it may omit widget identifiers, temporal dependencies, or navigation context. We later compare this recall metric with a precision-based alternative in our metric analysis (§4).

V. Iterative Refinement and Final Verification. If $R(A, B) < R_{\min}$ or execution fails, I is refined by injecting missing context and re-executed to obtain an updated **Execution Sequence**.¹ This replay-refine loop continues until alignment improves ($R(A, B) \geq R_{\min}$) or the refinement budget $F_{\max}$ is reached. We set $R_{\min} = 0.7$ based on empirical validation balancing stability and coverage. When I yields stable, deterministic execution, the pair (I, B) is accepted as a **Verified Trajectory**. Otherwise, it is discarded, and its misalignment contributes to the **hardness reward** for future exploration.

Only verified trajectories that pass alignment verification are admitted to the corpus, ensuring that synthesized data are both **reliable** (free from misaligned supervision) and **informative** (rich in semantic-ambiguous yet executable

¹The prompt templates used at each stage are provided in Appendix C.

Table 2. WebArena evaluation results across website domains. All values are success rates (%).

Base Model	Strategies	WebArena (Success Rate %)			
		Gitlab	Maps	Reddit	Overall
GPT-4o [20]	Zero-Shot [63]	32.14 N.A.	6.42 N.A.	27.36 N.A.	22.82 N.A.
InternVL2-4B [4]	Zero-Shot [63]	1.11 -95.1%	0.00 -100.0%	0.94 -94.8%	0.76 -96.3%
	Task-Driven [25]	9.52 -58.1%	5.00 -75.2%	0.00 -100.0%	5.72 -72.2%
	Task-Driven w. Self-Instruct [62]	9.52 -58.1%	7.50 -67.0%	0.00 -100.0%	6.41 -68.9%
	OS-Genesis [59]	7.94 -65.1%	7.50 -67.0%	3.13 -82.6%	6.53 -68.3%
HATS		22.73 0.0%	20.19 0.0%	17.92 0.0%	20.60 0.0%
InternVL2-8B [4]	Zero-Shot [63]	1.37 -95.1%	0.95 -96.2%	0.97 -95.2%	1.13 -95.5%
	Task-Driven [25]	6.35 -77.2%	2.50 -89.9%	0.00 -100.0%	3.58 -85.6%
	Task-Driven w. Self-Instruct [62]	7.94 -71.4%	0.00 -100.0%	6.25 -69.0%	5.30 -78.7%
	OS-Genesis [59]	6.35 -77.2%	10.00 -59.6%	9.34 -53.7%	8.16 -67.2%
HATS		27.81 0.0%	24.77 0.0%	20.19 0.0%	24.87 0.0%
Qwen2-VL-7B [61]	Zero-Shot [63]	1.14 -95.4%	0.98 -95.4%	1.96 -91.4%	1.32 -94.3%
	Task-Driven [25]	6.35 -74.6%	5.00 -76.3%	6.25 -72.6%	5.95 -74.4%
	Task-Driven w. Self-Instruct [62]	4.84 -80.6%	7.50 -64.5%	3.13 -86.3%	5.12 -78.0%
	OS-Genesis [59]	15.87 -36.5%	5.00 -76.4%	15.63 -31.3%	12.81 -44.9%
HATS		25.00 0.0%	21.10 0.0%	22.77 0.0%	23.28 0.0%

behaviors). The effectiveness of the refinement module is analyzed in §4, where multi-round replay is shown to monotonically increase action-level recall and substantially improve downstream task success. As visualized in Fig. 5, iterative refinement transforms vague one-shot instructions into precise and executable task instructions, bridging the gap between textual goals and GUI-level actions.

3.3. Closed-Loop Integration

This stage corresponds to the **Backpropagation** phase of HD-MCTS, where the alignment scores of refined trajectories are converted into a unified hardness feedback signal that guides subsequent exploration. In contrast to conventional MCTS, which backpropagates environment-provided task rewards, GUI trajectory synthesis exposes no such scalar supervision. We therefore propagate an **alignment-derived hardness reward** that quantifies semantic difficulty and serves as a task-adaptive surrogate objective.

I. Motivation and Design of the Hardness Reward. The core idea is to turn semantic misalignment, typically noisy supervision, into informative feedback that prioritizes ambiguous or hard regions of the GUI state space. During refinement, we compute the **action-level reconstruction recall** $R(A, B)$ (Eq. 2), which measures how well a synthesized instruction I reproduces its reference sequence A . High recall indicates clear semantic grounding, whereas low recall reflects ambiguous visual cues, missing preconditions, or unstable language–interface mappings. Thus, R naturally serves as an empirical estimate of **task hardness**.

Building on this insight, we define the **hardness reward** as an inverse function of $R(A, B)$:

$$r(A, B) = (R(A, B) + \epsilon)^{-\alpha}, \quad \epsilon > 0, \alpha > 0. \quad (3)$$

This formulation (i) yields a continuous, bounded score that increases smoothly as alignment deteriorates, and (ii) di-

rectly links exploration to the weakest regions of instruction–execution understanding. We additionally clip hardness into a stable range (e.g., $[0, H_{\max}]$) to avoid extreme values. Within HD-MCTS, this reward replaces task-return signals and encourages revisiting trajectories that are hard to align yet semantically valuable. We use the default configuration ($\epsilon=0.01$, $\alpha=1$), and analyze parameter sensitivity in §4.

II. Hardness Propagation. After refinement, each verified instruction–trajectory pair (I, B) yields a hardness value $r(A, B)$. We backpropagate this reward along the **tree-policy path** from the expansion node to the root:

$$N(v, a) \leftarrow N(v, a) + 1, \quad Q(v, a) \leftarrow Q(v, a) + \frac{r - Q(v, a)}{N(v, a)}. \quad (4)$$

Following standard MCTS, only nodes selected by the tree policy receive credit. Since hardness characterizes the entire instruction–trajectory pair rather than any single action, the same scalar reward is propagated along the path. These updates reshape the value estimates, biasing future UCB selection (Eq. 1) toward semantically challenging regions.

In summary, the closed loop of **HATS** turns semantic ambiguity from a training liability into a useful learning signal. Misalignment during replay generates the hardness reward, which in turn guides exploration toward richer and harder-to-align interactions. **Verified trajectories** annotated with hardness form a semantically grounded dataset for downstream SFT training. We follow the two-stage paradigm of OS-GENESIS [59] for planning-level and action-level learning, with hardness-aware weighting detailed in Appendix B.

4. Experiments

We evaluate **HATS** on two representative GUI benchmarks: ANDROIDWORLD [48] and WEBARENA [78], which together span diverse mobile and web interaction patterns. On ANDROIDWORLD, all 116 tasks are grouped into four func-

Table 3. Parameter sensitivity of the hardness reward in Eq. 3.

Hardness Metric	Hyperparameters ϵ α		Success Rate (%)
$(R(A, B) + \epsilon)^{-\alpha}$	0.01	0.50	22.7 -55.3%
	0.01	1.00	50.8 0.0%
	0.01	2.00	39.2 -22.8%
	0.10	0.50	13.8 -72.8%
	0.10	1.00	40.0 -21.3%
	0.10	2.00	37.9 -25.3%

tional categories, **DLS**, **S&C**, **S&U**, and **P&W**, defined by their interaction intent (see caption of Table 1), with the full mapping provided in Appendix B.1. For WEBARENA, we evaluate three lightweight yet representative domains (**Gitlab**, **Maps**, **Reddit**) to reduce computational cost.

Implementation Details. Following [59], we use three VLM backbones (InternVL2-4B/8B [4] and Qwen2-VL-7B [61]), with **GPT-4o** [20] for instruction synthesis and reward modeling. All methods are trained under the same SFT setting with screenshots and **a11ytree** inputs. We compare **HATS** against four representative data-synthesis paradigms: ZERO-SHOT (CoT + M3A) [48], TASK-DRIVEN GENERATION [25], SELF-INSTRUCT [62], and OS-GENESIS [59]. **HATS** uses the same 1K-trajectory synthesis budget as OS-GENESIS; the additional synthesis cost is bounded and tunable, with HD-MCTS controlled by the iteration budget and replay-refine capped by $F_{\max}$ with early stopping. Full training and efficiency details are provided in Appendix B.

Overall Performance.² Tables 1 and 2 show that **HATS** achieves the highest success rates across all backbones. Improvements are especially notable in categories where existing methods perform poorly. For example, on ANDROIDWORLD **S&C**, OS-GENESIS reaches only 11.11% on InternVL2-4B and 0.00% on Qwen2-VL-7B, while **HATS** attains 38.46% and 30.77%. Similar gains appear on **P&W** (e.g., 1.85% vs. 9.88% on InternVL2-4B) and on WEBARENA domains such as **Gitlab** (7.94% vs. 22.73%). During baseline reproduction, we also observed that adding SELF-INSTRUCT data sometimes degraded TASK-DRIVEN performance due to instruction-execution misalignment and noisy supervision, further illustrating the sensitivity of GUI agents to data quality. Following [59], the GPT-4o-based M3A agent is reported only as a reference baseline. These representative cases highlight a consistent pattern: **HATS** substantially boosts performance precisely in interaction regimes that existing data-synthesis pipelines struggle with, leading to the highest overall success rates on both mobile and web environments.

Component Analysis.³ We next analyze the effectiveness

²Baseline reproduction uses available resources including official checkpoints, released data, and our own reimplementations (see Appendix B).

³To save computation, we evaluate on a representative subset of ANDROIDWORLD; see Appendix B.5.

Table 4. Metric comparison of action-level alignment.

Alignment Metric Definition in Eq. 2	Success Rate (%)
$\frac{1}{ A_a } \sum_{a \in A_a} \mathbf{1}[\exists b \in B_a : \text{match}(a, b s)]$	50.8
$\frac{1}{ B_a } \sum_{b \in B_a} \mathbf{1}[\exists a \in A_a : \text{match}(a, b s)]$	36.0

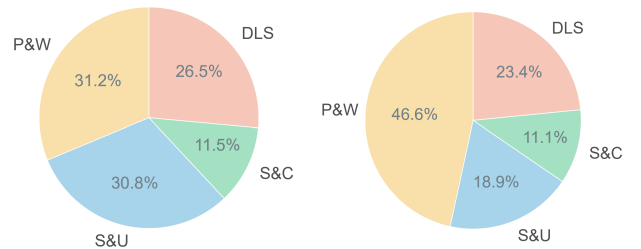
of the three components of **HATS**: (i) the alignment-based hardness metric, (ii) hardness-driven exploration, and (iii) alignment-guided refinement.

Analysis I. Hardness Metric Validation. We validate the hardness definition in Eq. 3 through two complementary analyses: parameter sensitivity and metric substitution.

- **Parameter sensitivity.** We vary ϵ and α in Eq. 3 and observe consistent trends (Table 3). Large ϵ flattens hardness differences, while small or large α distort the reward landscape and reduce discriminative power. The default configuration ($\epsilon=0.01$, $\alpha=1$) offers a balanced scaling that preserves sensitivity to semantic hardness while maintaining stable reward behavior.
- **Metric comparison.** We substitute recall in Eq. 2 with an action-level precision metric and derive an analogous hardness mapping. As shown in Table 4, recall-based hardness correlates substantially better with downstream performance: recall captures coverage of intended actions, which is the main failure mode in ambiguous GUI workflows; precision does not penalize missing steps.



(a) Performance comparison on ANDROIDWORLD with InternVL2-4B.



(b) Distribution: Baseline [62]

(c) Distribution: Ours

Figure 6. **Performance comparison & corpus composition.** Top: performance comparison on ANDROIDWORLD using InternVL2-4B. Bottom: corpus composition by task category for the baseline (left) and **HATS** (right), where **HATS** increases exploration in categories that are empirically harder for existing methods (e.g., **P&W**), while reducing exploration in easier categories (e.g., **S&U**).

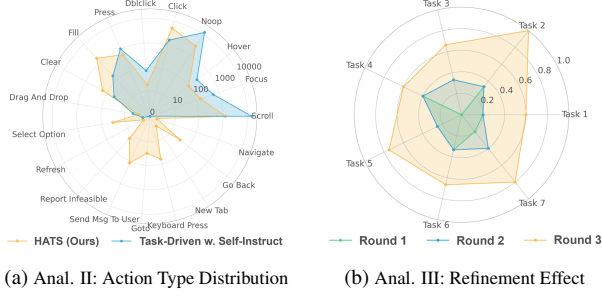


Figure 7. **Radar chart comparison.** (a) Action-type distribution on WEBARENA, showing distributional differences between the baseline and **HATS**. (b) Per-task view of multi-round refinement, where the shaded area expands across rounds, indicating higher action-level recall and better synthesized trajectories.

Analysis II. Effectiveness of Hardness-Driven Exploration. We use two qualitative analyses to show that hardness-driven exploration indeed shifts sampling coverage toward semantically valuable, ambiguity-rich regions.

- **Distributional coverage analyses.** Fig. 6a reveals a clear pattern across baselines: tasks in **S&U** exhibit uniformly high success rates, whereas **P&W** tasks show consistently lower success, suggesting that **P&W** contains a larger fraction of semantically complex, multi-step workflows. Motivated by this observation, we compare category proportions in the synthesized trajectories (Fig. 6 (b-c)). Relative to the baseline⁴, **HATS** increases the sampling weight of **P&W** from **31.2%** to **46.6%**, while reducing **S&U** from **30.8%** to **18.9%**. This redistribution aligns with the empirical hardness indicated in Fig. 6a: harder categories receive more coverage, while easier ones are deemphasized. A similar shift is observed in the action-type distributions on WEBARENA (Fig. 7a), which likely contributes to its improved performance.
- **Semantics-focused auditing.** We further compare whether the final trajectories synthesized by **HATS** contain more semantic ambiguity than those produced by **OS-GENESIS**. Using a third-party LLM auditor (**GPT-4o**) following our rubric in Appendix B.5, each trajectory is independently evaluated for three ambiguity types: context dependency, sequential dependency, and visual ambiguity. As shown in Fig. 8, a consistently larger fraction of **HATS** trajectories receive ambiguity labels across all three categories, indicating that hardness-driven exploration uncovers interactions where language–action grounding is intrinsically challenging.

Analysis III. Effectiveness of Alignment-Guided Refinement. We evaluate how multi-round replay–refinement improves instruction–execution alignment.

⁴OS-Genesis (10/2025) does not provide category-labeled training data; we thus use TASK-DRIVEN W. SELF-INSTR. for category-level comparison.

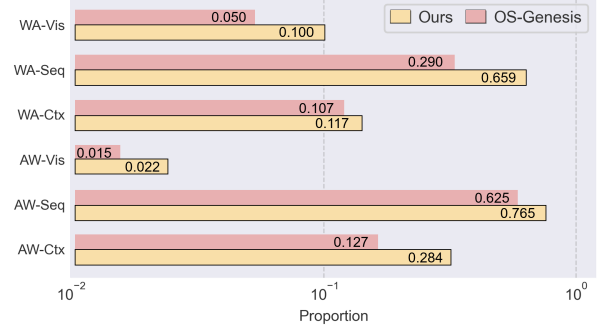


Figure 8. **Semantic-ambiguity ratios at log scale.** **HATS** vs. **OS-GENESIS** across ambiguity types Vis, Seq, and Ctx on ANDROIDWORLD (AW) and WEBARENA (WA). Bars correspond to AW/WA-Ctx/Seq/Vis pairs. Trajectories may involve multiple ambiguity types, so categories are not mutually exclusive.

- **Overall effect.** Table 5 shows that, refinement yields substantial gains: the average action-level reconstruction recall $R(A, B)$ (Eq. 2) increases from **0.26** to **0.40**, accompanied by a corresponding improvement in downstream success rate (from **16.00%** to **23.07%**). This indicates that iterative replay increasingly corrects mismatches and strengthens overall instruction fidelity.
- **Single-case insight.** A per-task view in Fig. 7b, using seven representative tasks (Appendix D), exhibits the same steadily improving trend: the radar area expands monotonically across rounds, showing that iterative correction progressively fills missing context, resolves sequential inconsistencies, and produces more executable task instructions for individual behaviors.

Table 5. **Ablation on multi-round refinement on AndroidWorld.**

Refinement Rounds	R_{avg}	Success Rate (%)
One-Shot	0.26 -35.0%	16.00 -30.6%
Round 1	0.19 -52.5%	15.56 -32.6%
Round 2	0.29 -27.5%	21.43 -7.1%
Round 3	0.40 0.0%	23.07 0.0%

5. Conclusion

We introduced **HATS**, a closed-loop framework that transforms instruction–execution feedback into a hardness signal to guide trajectory synthesis for GUI agents. Through the integration of a **hardness-driven exploration** module and an **alignment-guided refinement** module, the framework jointly enhances the diversity of semantic-ambiguous actions and the semantic consistency of generated data. By promoting exploration toward challenging yet informative interactions and ensuring alignment at the instruction–trajectory level, **HATS** produces higher-quality, more learnable datasets that enable GUI agents to generalize more robustly across real-world interfaces.

6. Acknowledgement

This research is supported by the National Research Foundation, Singapore under its AI Singapore Programme (AISG Award No: AISG2-PhD-2023-08-042T-J), the National Natural Science Foundation of China (Grant No. 62306090 and Grant No. U24A20328), the China Postdoctoral Science Foundation (Grant No. 2024M764192), the Natural Science Foundation of Guangdong Province of China (Grant No. 2024A1515010147), the Natural Science Foundation of Shenzhen City of China (Grant No. JCYJ20250604145700001), and the Beijing Natural Science Foundation (Grant No. L254018), and is part of the programme DesCartes, which is supported by the National Research Foundation, Prime Minister's Office, Singapore under its Campus for Research Excellence and Technological Enterprise (CREATE) programme.

References

- [1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *ICML*, 2009. 1
- [2] Yuxiang Chai, Siyuan Huang, Yazhe Niu, Han Xiao, Liang Liu, Dingyu Zhang, Peng Gao, Shuai Ren, and Hongsheng Li. Amex: Android multi-annotation expo dataset for mobile gui agents. In *ACL*, 2024. 1
- [3] Wentong Chen, Junbo Cui, Jinyi Hu, Yujia Qin, Junjie Fang, Yue Zhao, Chongyi Wang, Jun Liu, Guirong Chen, Yupeng Huo, Yuan Yao, Yankai Lin, Zhiyuan Liu, and Maosong Sun. Guicourse: From general vision language models to versatile gui agents. In *ACL*, 2025. 1
- [4] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, Bin Li, Ping Luo, Tong Lu, Yu Qiao, and Jifeng Dai. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *CVPR*, 2024. 5, 6, 7, 3
- [5] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *ACL*, 2024. 2, 1
- [6] Kanzhi Cheng, Yantao Li, Fangzhi Xu, Jianbing Zhang, Hao Zhou, and Yang Liu. Vision-language models can self-improve reasoning via reflection. In *ACL*, 2025. 1
- [7] Thibault Le Sellier de Chezelles, Maxime Gasse, Alexandre Lacoste, Massimo Caccia, Alexandre Drouin, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omid Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Graham Neubig, Quentin Cappart, Russ Salakhutdinov, and Nicolas Chapados. The browsergym ecosystem for web agent research. In *TMLR*, 2025. 1
- [8] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschman, Daniel Afegan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. Rico: A mobile app dataset for building data-driven design applications. In *UIST*, 2017. 3, 1
- [9] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *NeurIPS*, 2023. 1
- [10] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In *ICML*, 2024. 1
- [11] Zane Durante, Qiuyuan Huang, Naoki Wake, Ran Gong, Jae Sung Park, Bidipta Sarkar, Rohan Taori, Yusuke Noda, Demetri Terzopoulos, Yejin Choi, et al. Agent ai: Surveying the horizons of multimodal interaction. *arXiv preprint arXiv:2401.03568*, 2024. 2, 1
- [12] Tao Feng, Chuanyang Jin, Jingyu Liu, Kunlun Zhu, Haoqin Tu, Zirui Cheng, Guanyu Lin, and Jiaxuan You. How far are we from agi. *CoRR*, 2024. 2, 1
- [13] Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents. In *ICML*, 2018. 1
- [14] Ruize Gao, Feng Liu, Jingfeng Zhang, Bo Han, Tongliang Liu, Gang Niu, and Masashi Sugiyama. Maximum mean discrepancy test is aware of adversarial attacks. In *ICML*, 2021. 1
- [15] Ruize Gao, Jiong Xiao Wang, Kaiwen Zhou, Feng Liu, Binghui Xie, Gang Niu, Bo Han, and James Cheng. Fast and reliable evaluation of adversarial robustness with minimum-margin attack. In *ICML*, 2022. 1
- [16] Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for gui agents. In *ICLR*, 2025. 1
- [17] Alex Graves, Marc G Bellemare, Jacob Menick, Remi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. In *ICML*, 2017. 1
- [18] Zhiting Hu and Tianmin Shu. Language models, agent models, and world models: The law for machine reasoning and planning. In *NeurIPS*, 2023. 1
- [19] Keke Huang, Ruize Gao, Bogdan Cautis, and Xiaokui Xiao. Scalable continuous-time diffusion framework for network inference and influence estimation. In *WWW*, 2024. 1
- [20] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 5, 6, 7, 3
- [21] Chengyou Jia, Minnan Luo, Zhuohang Dang, Qiushi Sun, Fangzhi Xu, Junlin Hu, Tianbao Xie, and Zhiyong Wu. Agentstore: Scalable integration of heterogeneous agents as specialized generalist computer assistant. In *ICLR*, 2024. 1
- [22] Chuanyang Jin, Yutong Wu, Jing Cao, Jiannan Xiang, Yen-Ling Kuo, Zhiting Hu, Tomer Ullman, Antonio Torralba, Joshua B Tenenbaum, and Tianmin Shu. Mmtom-qa: Multimodal theory of mind question answering. In *ACL*, 2024. 1
- [23] Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem Alshikh, and Ruslan Salakhutdinov. Omniact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In *ECCV*, 2024. 2, 1

- [24] M Kumar, Benjamin Packer, and Daphne Koller. Self-paced learning for latent variable models. In *NeurIPS*, 2010. 1
- [25] Hanyu Lai, Xiao Liu, Iat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, and Jie Tang. Autowebglm: A large language model-based web navigating agent. In *KDD*, 2024. 2, 5, 6, 7, 3
- [26] Hao Li, Qi Lv, Rui Shao, Xiang Deng, Yinchuan Li, Jianye Hao, and Liqiang Nie. Star: Learning diverse robot skill abstractions through rotation-augmented vector quantization. In *ICML*, 2025. 1
- [27] Wei Li, William E Bishop, Alice Li, Christopher Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on UI control agents. In *NeurIPS*, 2024. 2, 3, 1
- [28] Wei Li, Renshan Zhang, Rui Shao, Jie He, and Liqiang Nie. Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification. In *NeurIPS*, 2025. 1
- [29] Wei Li, Jizhihui Liu, Yixing Li, Junwen Tong, Rui Shao, and Liqiang Nie. Consisvla-4d: Advancing spatiotemporal consistency in efficient 3d-perception and 4d-reasoning for robotic manipulation. In *CVPR*, 2026.
- [30] Wei Li, Renshan Zhang, Rui Shao, Zhijian Fang, Kaiwen Zhou, Zhuotao Tian, and Liqiang Nie. Semanticvla: Semantic-aligned sparsification and enhancement for efficient robotic manipulation. In *AAAI*, 2026.
- [31] Zaijing Li, Bing Hu, Rui Shao, Gongwei Chen, Dongmei Jiang, Pengwei Xie, Jianye Hao, and Liqiang Nie. Global prior meets local consistency: Dual-memory augmented vision-language-action model for efficient robotic manipulation. In *CVPR*, 2026.
- [32] Zaijing Li, Bing Hu, Rui Shao, Gongwei Chen, Dongmei Jiang, Pengwei Xie, Jianye Hao, and Liqiang Nie. Global prior meets local consistency: Dual-memory augmented vision-language-action model for efficient robotic manipulation. *arXiv preprint arXiv:2602.20200*, 2026. 1
- [33] Musen Lin, Minghao Liu, Taoran Lu, Lichen Yuan, Yiwei Liu, Haonan Xu, Yu Miao, Yuhao Chao, and Zhaojian Li. Gui-rewalk: Massive data generation for gui agent via stochastic exploration and intent-aware reasoning. *arXiv preprint arXiv:2509.15738*, 2025. 3
- [34] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, and Percy Liang. Reinforcement learning on web interfaces using workflow-guided exploration. In *ICLR*, 2018. 1
- [35] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019. 3
- [36] Quanfeng Lu, Wenqi Shao, Zitao Liu, Fanqing Meng, Boxuan Li, Botong Chen, Siyuan Huang, Kaipeng Zhang, Yu Qiao, and Ping Luo. Gui odyssey: A comprehensive dataset for cross-app gui navigation on mobile devices. In *ICCV*, 2025. 1
- [37] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. Weblinx: Real-world website navigation with multi-turn dialogue. In *ICML*, 2024. 2, 3, 1
- [38] Qi Lv, Hao Li, Xiang Deng, Rui Shao, Yinchuan Li, Jianye Hao, Longxiang Gao, Michael Yu Wang, and Liqiang Nie. Spatial-temporal graph diffusion policy with kinematic modeling for bimanual robotic manipulation. In *CVPR*, pages 17394–17404, 2025. 1
- [39] Shikhar Murty, Christopher D Manning, Peter Shaw, Mandar Joshi, and Kenton Lee. BAGEL: Bootstrapping agents by guiding exploration with language. In *ICML*, 2024. 2, 3
- [40] Shikhar Murty, Dzmitry Bahdanau, and Christopher D Manning. Nnetscape navigator: Complex demonstrations for web agents without a demonstrator. In *ICLR*, 2025. 1
- [41] Runliang Niu, Jindong Li, Shiqi Wang, Yali Fu, Xiyu Hu, Xueyuan Leng, He Kong, Yi Chang, and Qi Wang. Screenagent: A vision language model-driven computer control agent. In *IJCAI*, 2024. 1
- [42] Vardaan Pahuja, Yadong Lu, Corby Rosset, Boyu Gou, Arindam Mitra, Spencer Whitehead, Yu Su, and Ahmed Hassan. Explorer: Scaling exploration-driven web trajectory synthesis for multimodal web agents. In *ACL Findings*, 2025. 1
- [43] Jiayi Pan, Yichi Zhang, Nicholas Tomlin, Yifei Zhou, Sergey Levine, and Alane Suhr. Autonomous evaluation and refinement of digital agents. In *COLM*, 2024. 1
- [44] Ajay Patel, Markus Hofmarcher, Claudiu Leoveanu-Condrei, Marius-Constantin Dinu, Chris Callison-Burch, and Sepp Hochreiter. Large language models can self-improve at web agent tasks. *arXiv preprint arXiv:2405.20309*, 2024. 2, 3
- [45] Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep rl: A short survey. In *IJCAI*, 2020. 1
- [46] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *KDD*, 2020. 3
- [47] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy P Lillicrap. Androidinthewild: A large-scale dataset for android device control. In *NeurIPS*, 2023. 1
- [48] Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, et al. Androidworld: A dynamic benchmarking environment for autonomous agents. In *ICLR*, 2025. 3, 6, 7, 1
- [49] Rui Shao, Xiangyuan Lan, and Pong C Yuen. Deep convolutional dynamic texture learning with adaptive channel-discriminability for 3d mask face anti-spoofing. In *IJCB*, 2017. 1
- [50] Rui Shao, Xiangyuan Lan, Jiawei Li, and Pong C Yuen. Multi-adversarial discriminative deep domain generalization for face presentation attack detection. In *CVPR*, 2019. 1
- [51] Rui Shao, Tianxing Wu, and Ziwei Liu. Detecting and grounding multi-modal media manipulation. In *CVPR*, 2023. 1
- [52] Rui Shao, Tianxing Wu, Jianlong Wu, Liqiang Nie, and Ziwei Liu. Detecting and grounding multi-modal media manipulation and beyond. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [53] Rui Shao, Wei Li, Lingsen Zhang, Renshan Zhang, Zhiyang Liu, Ran Chen, and Liqiang Nie. Large vlm-based vision-language-action models for robotic manipulation: A survey. *arXiv preprint arXiv:2508.13073*, 2025. 1

- [54] Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. World of bits: An open-domain platform for web-based agents. In *ICML*, 2017. 3, 1
- [55] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2024. 1
- [56] Abhinav Shrivastava, Abhinav Gupta, and Ross Girshick. Training region-based object detectors with online hard example mining. In *CVPR*, 2016. 1
- [57] Qiushi Sun, Zhangyue Yin, Xiang Li, Zhiyong Wu, Xipeng Qiu, and Lingpeng Kong. Corex: Pushing the boundaries of complex reasoning through multi-model collaboration. In *COLM*, 2023. 1
- [58] Qiushi Sun, Zhirui Chen, Fangzhi Xu, Kanzhi Cheng, Chang Ma, Zhangyue Yin, Jianing Wang, Chengcheng Han, Renyu Zhu, Shuai Yuan, et al. A survey of neural code intelligence: Paradigms, advances and beyond. *arXiv preprint arXiv:2403.14734*, 2024. 2, 1
- [59] Qiushi Sun, Kanzhi Cheng, Zichen Ding, Chuanyang Jin, Yian Wang, Fangzhi Xu, Zhenyu Wu, Chengyou Jia, Liheng Chen, Zhoumianze Liu, et al. Os-genesis: Automating gui agent trajectory construction via reverse task synthesis. In *ACL*, 2025. 2, 3, 4, 5, 6, 7, 1
- [60] Junyang Wang, Haiyang Xu, Jiabo Ye, Ming Yan, Weizhou Shen, Ji Zhang, Fei Huang, and Jitao Sang. Mobile-agent: Autonomous multi-modal mobile device agent with visual perception. In *ICLR Workshop on LLM Agents*, 2024. 2, 1
- [61] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 5, 6, 7, 3
- [62] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In *ACL*, 2023. 5, 6, 7, 3
- [63] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022. 5, 6, 3
- [64] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. In *COLM*, 2023. 2, 1
- [65] Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. OS-copilot: Towards generalist computer agents with self-improvement. In *ICLR*, 2024. 2, 1
- [66] Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, et al. Os-atlas: A foundation action model for generalist gui agents. In *ICLR*, 2025. 2, 1
- [67] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS*, 2024. 1
- [68] Fangzhi Xu, Qiushi Sun, Kanzhi Cheng, Jun Liu, Yu Qiao, and Zhiyong Wu. Interactive evolution: A neural-symbolic self-training framework for large language models. In *ACL*, 2025. 1
- [69] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. In *NeurIPS*, 2022. 1
- [70] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023. 4
- [71] Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, and Jie Tang. AgentTuning: Enabling generalized agent abilities for LLMs. In *ACL Findings*, 2024. 1
- [72] Chi Zhang, Zhao Yang, Jiaxuan Liu, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. Appagent: Multimodal agents as smartphone users. In *CHI*, 2025. 1
- [73] Jianguo Zhang, Tian Lan, Rithesh Murthy, Zhiwei Liu, Weiran Yao, Ming Zhu, Juntao Tan, Thai Hoang, Zuxin Liu, Liangwei Yang, et al. Agentohana: Design unified data and training pipeline for effective agent learning. *arXiv preprint arXiv:2402.15506*, 2024. 1
- [74] Jiwen Zhang, Jihao Wu, Yihua Teng, Minghui Liao, Nuo Xu, Xiao Xiao, Zhongyu Wei, and Duyu Tang. Android in the zoo: Chain-of-action-thought for gui agents. In *EMNLP Findings*, 2024. 1
- [75] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *ICML*, 2024. 1
- [76] Longtao Zheng, Zhiyuan Huang, Zhenghai Xue, Xinrun Wang, Bo An, and Shuicheng Yan. Agentstudio: A toolkit for building general virtual agents. In *ICLR*, 2024. 1
- [77] Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *ICLR*, 2024. 2
- [78] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *ICLR*, 2024. 3, 6, 1
- [79] Xurui Zhou, Gongwei Chen, Yuquan Xie, Zajing Li, Kaiwen Zhou, Shuai Wang, Shuo Yang, Zhuotao Tian, and Rui Shao. Hiconagent: History context-aware policy optimization for gui agents. In *CVPR*, 2026. 1
- [80] Yijie Zhu, Rui Shao, Ziyang Liu, Jie He, Jizhihui Liu, Jiuru Wang, and Zitong Yu. H-gar: A hierarchical interaction framework via goal-driven observation-action refinement for robotic manipulation. In *AAAI*, 2026. 1